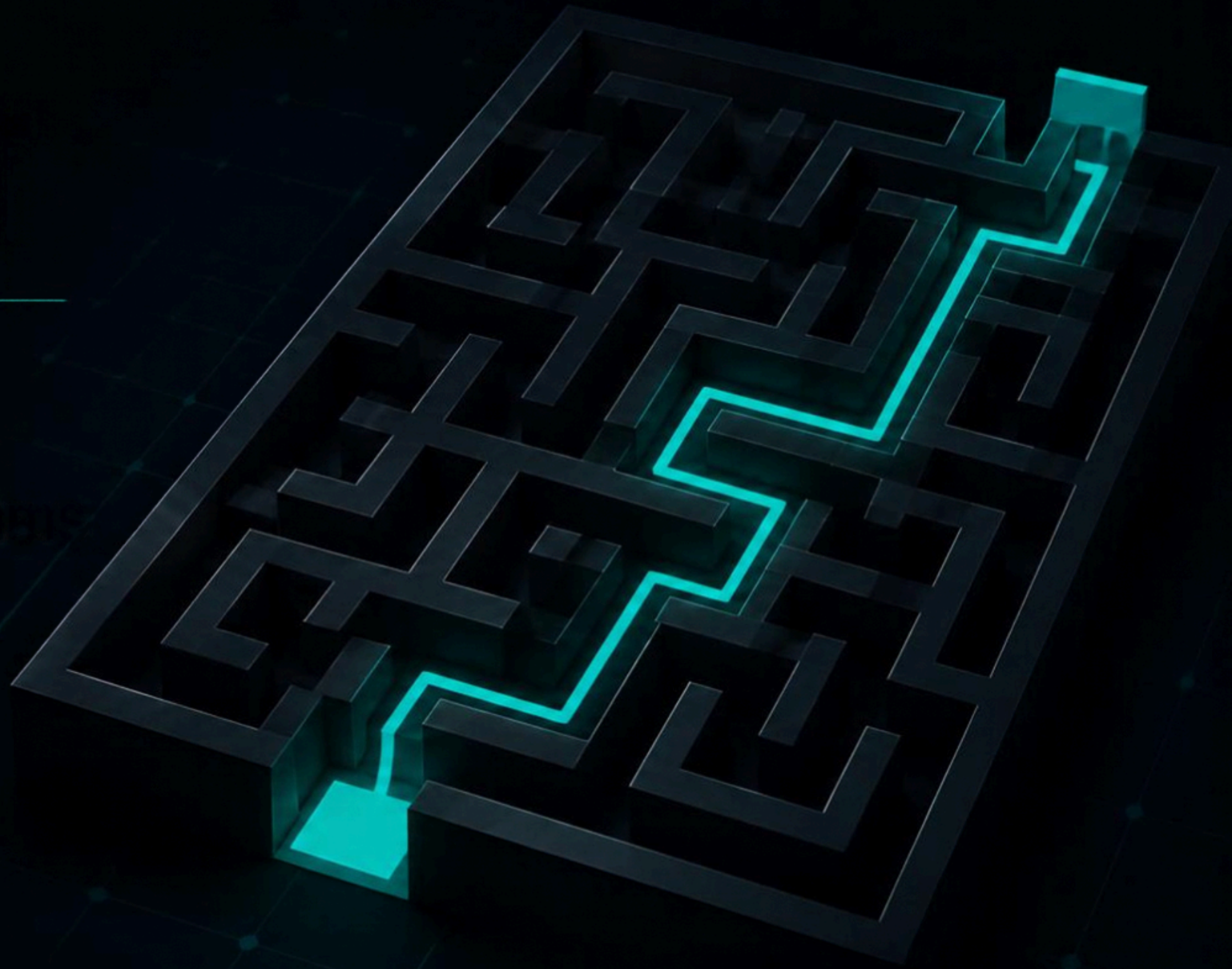
ABSTRACT



anai-powered maze solver visualizer for search algorithms

interactive exploration of maze solving

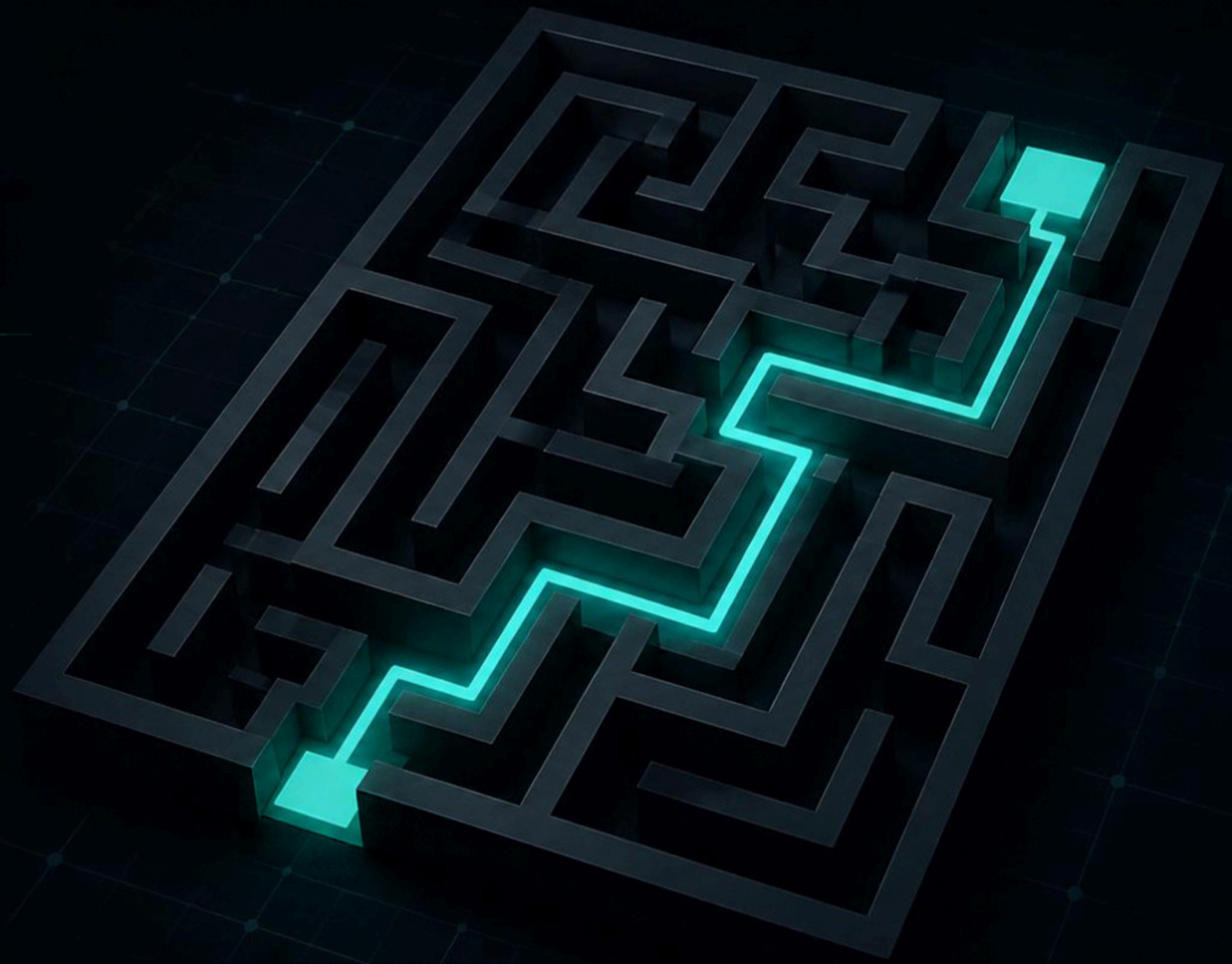
hill climbing and simulated annealing

compare search behavior step by step

TEAM
2116240701508 — Shri Sarvendran PS
2116240701469 — Sanjayram M
2116240701026 — Akhil kumaram S

PROBLEM STATEMENT

Search algorithms are abstract in theory
Students need to observe search behavior
A maze environment provides an intuitive demonstration



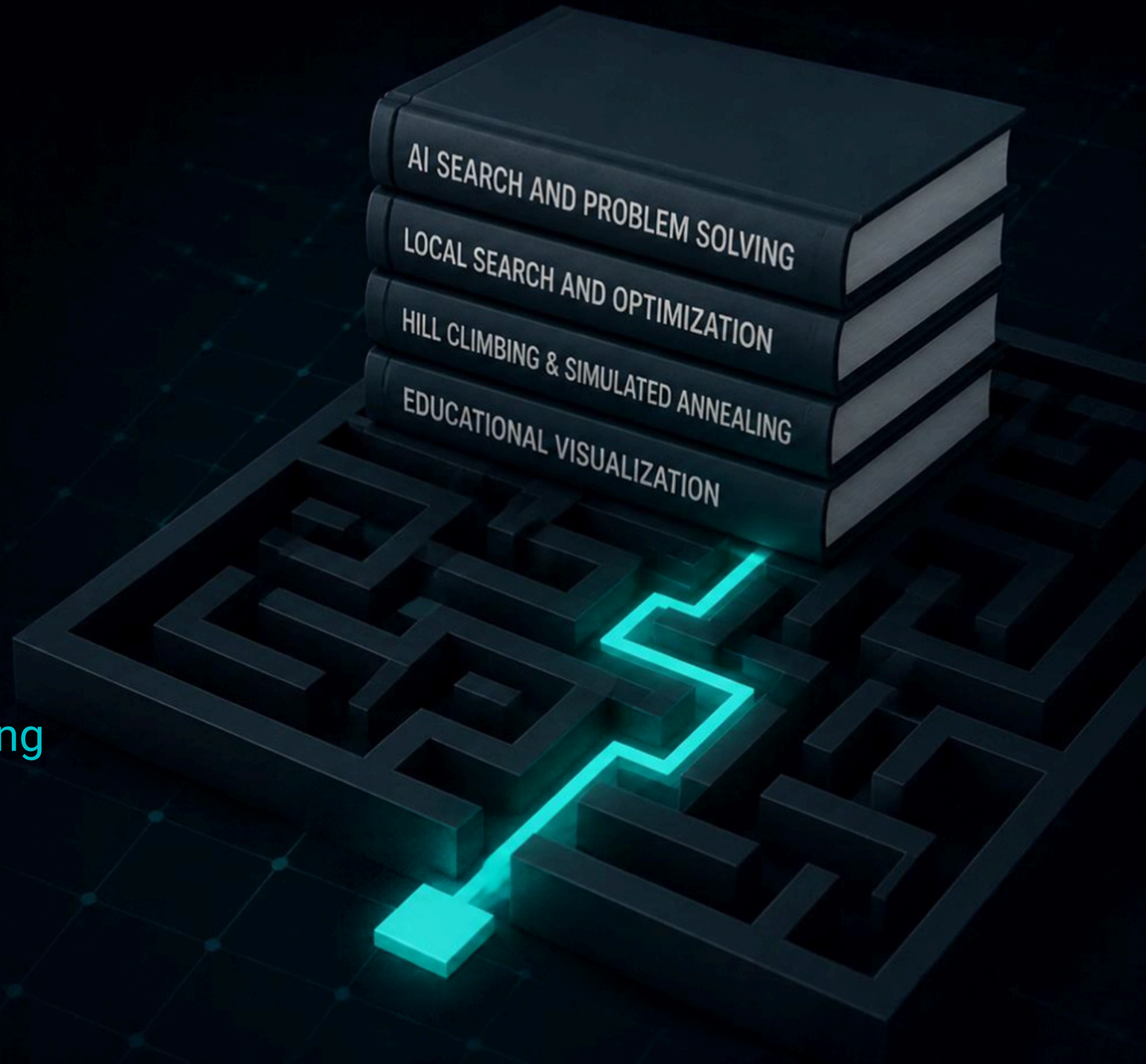
LITERATURE SURVEY

ai problem solving and search

local search and optimization

hill climbing and simulated annealing

educational visualization

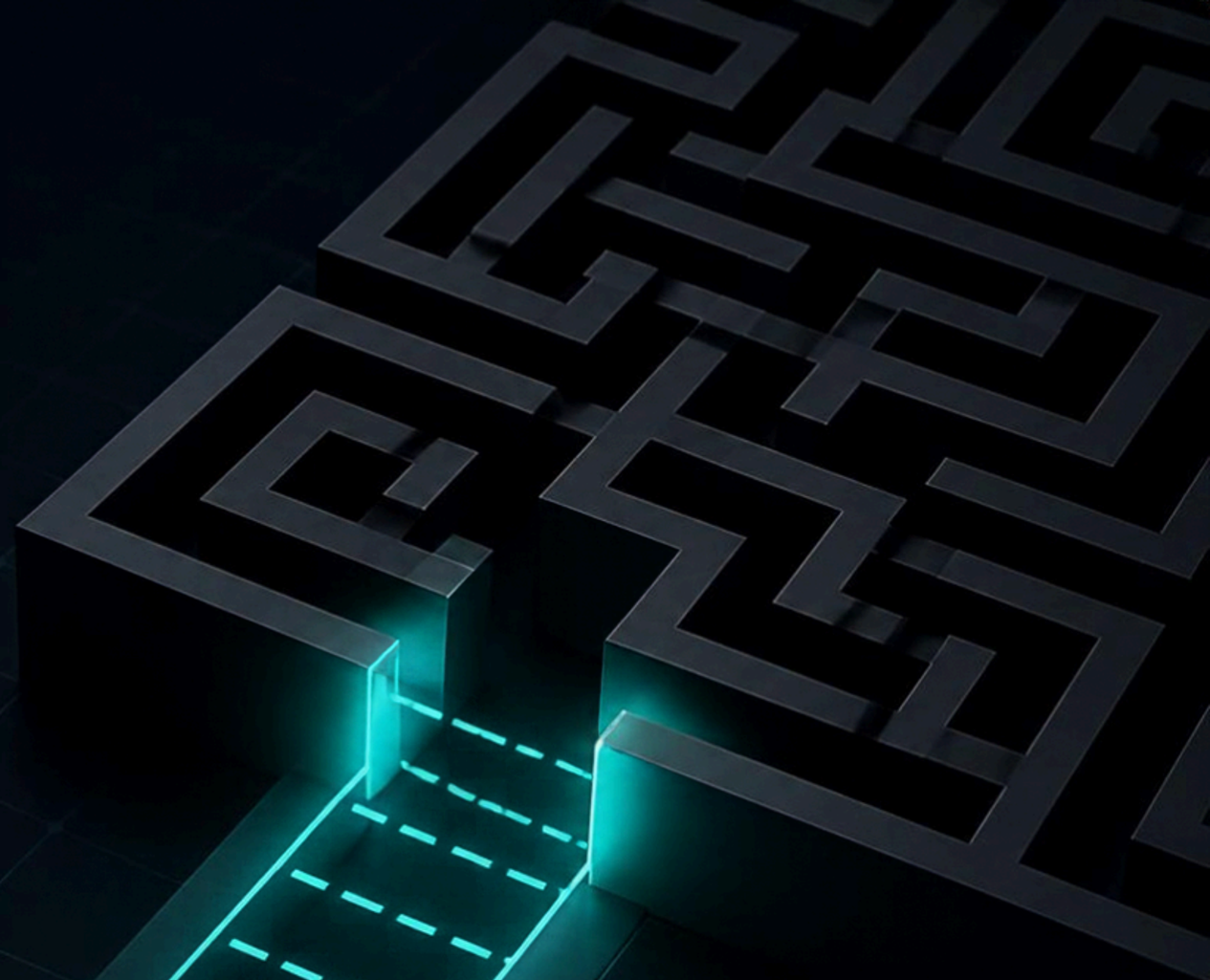


RESEARCH GAP

Search algorithmsare
taught in isolation

Few tools visualize them
side by side

Students need an
interactive, maze-focused
learning experience



RESEARCH OBJECTIVE

01	visualize search behavior
02	implement two local-search algorithms
03	compare behavior on the same maze
04	build an educational maze game



PROPOSED WORK

SYSTEM ARCHITECTURE



MAZE STATE

Represents the agent position within the maze

Transitions occur through valid neighboring cells

Goal state is the exit cell

HEURISTIC

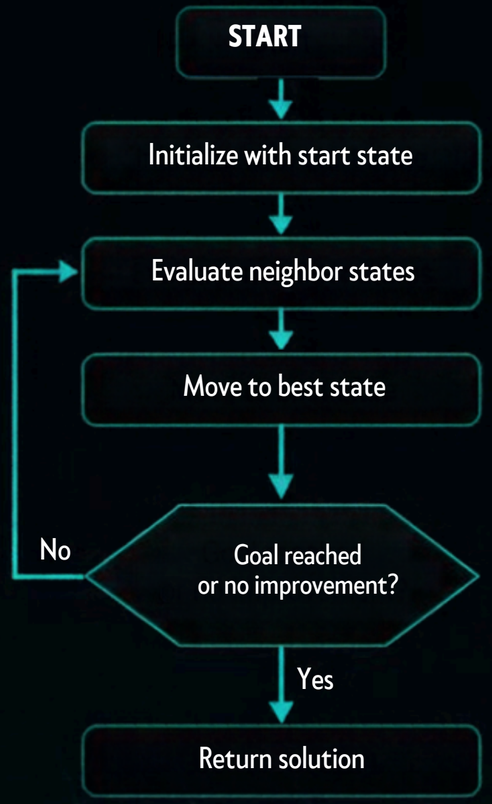
Uses Manhattan distance to the goal

Guides search toward promising states

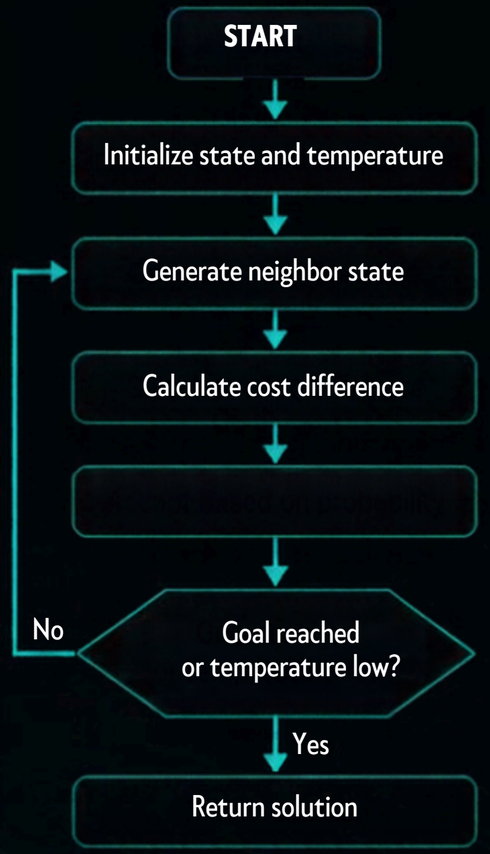
Reduces exploration and speeds convergence

ALGORITHM FLOWS

HILL CLIMBING FLOW



SIMULATED ANNEALING FLOW



IMPLEMENTATION

Programming language & tools

Python implementation using core libraries
for efficient algorithm execution and visualization

Maze / grid representation

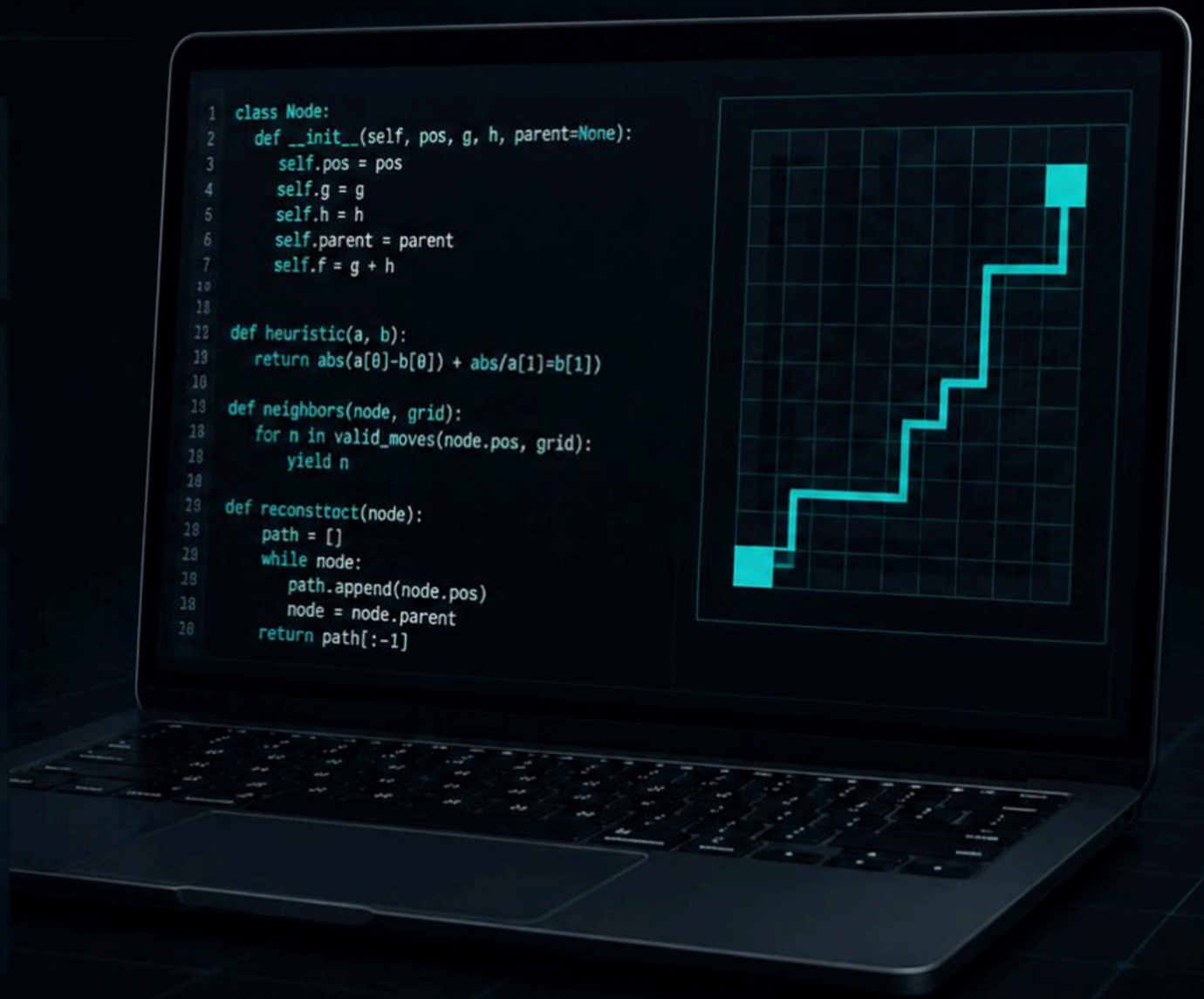
Two-dimensional grid structurewithwalls,
open cells, start, and goal states

Heuristic & search states

Eachstatetracksposition,cost,and parent
to guide and reconstruct the optimal path

System configuration

Lightweight setupforsmoothperformance and
interactive visualization



OUTPUT

Visualresultsfromthemaze solver

MAZE GENERATION



Randomly generated maze layout

VISITED CELLS



Cells explored during search process

SOLVED PATH



Optimal path from start to goal

ALGORITHM COMPARISON

	Path Length	Nodes Visited
BFS	21 : 28	124
DFS	24 : 6	312
A*	22 : 8	76
DIJKSTRA	22 : 8	102

Performance overview across algorithms

RESULT

Pathfinding behavior shows correct and efficient routes

Handles local optima without getting trapped

Execution observations confirm stable performance

Key metrics captured for each algorithm

